NAME: TANKISO MASOEBE

INTRO TO DATABASES WEEK13

1. SQL Statements Representing a Transaction

A typical e-commerce transaction (placing an order) involves:

```
-- Create a new order
INSERT INTO orders (order_id, customer_id, total_amount, status)
VALUES (101, 1, 500, 'PENDING');

-- Insert order items
INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (101, 10, 2, 250);

-- Reduce product inventory
UPDATE products
SET stock = stock - 2
WHERE product_id = 10;

-- Process payment (mark order as paid)
UPDATE orders
SET status = 'PAID'
WHERE order_id = 101;
```

2. Statements That May Cause Concurrency Problems

The most critical statement is:

```
UPDATE products
SET stock = stock - 2
WHERE product_id = 10;
```

Why this causes problems:

- If two users run this at the same time:
  - Both may read the same stock value.
  - Both subtract from it.
  - This results in a **Lost Update** or **overselling**.

Other risks:

- Order created but inventory not updated → **inconsistent data**
- Payment processed but order fails → **data integrity issue**

3. Transaction with ACID Properties

To ensure **Atomicity, Consistency, Isolation, and Durability**, use a transaction:

```
START TRANSACTION;

-- Lock the product row to prevent concurrent updates
SELECT stock
FROM products
WHERE product_id = 10
FOR UPDATE;

-- Check if enough stock is available (handled in application logic)

-- Create order
INSERT INTO orders (order_id, customer_id, total_amount, status)
VALUES (101, 1, 500, 'PENDING');

-- Insert order items
INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (101, 10, 2, 250);

-- Update inventory
UPDATE products
SET stock = stock - 2
WHERE product_id = 10;

-- Update order status after payment
UPDATE orders
SET status = 'PAID'
WHERE order_id = 101;

COMMIT;
```

If any step fails:

```
ROLLBACK;
```

4. Isolation Level Recommendation

SERIALIZABLE (Recommended)

Reason:

- Prevents:

    - Lost Updates
    - Dirty Reads
    - Non-repeatable Reads
    - Phantom Reads
- Ensures transactions behave as if executed one at a time.

Alternative:

- **REPEATABLE READ**
    - Better performance
    - Still prevents most issues, but may allow phantom reads depending on DBMS

5. Conclusion

- Inventory updates are the main source of concurrency problems.
- Transactions using START TRANSACTION, COMMIT, and ROLLBACK ensure **data integrity**.
- Using **SERIALIZABLE isolation level** provides maximum protection against concurrency issues in an e-commerce system.

2.Concurrency Problem Identification

1. Scenario 1

Situation:
A manager runs a report. During execution, an HR specialist updates an employee's salary. The report shows:

- Old salary on page 1
- New salary on page 2

Concurrency Problem: Non-Repeatable Read

Explanation:

- The same data (employee salary) is read more than once within a single transaction/report.
- Between reads, another transaction modifies the data.
- This causes inconsistent results within the same operation.

2. Scenario 2

Situation:
Two customers try to buy the last concert ticket at the same time.
Both see the ticket as available and both complete the purchase.

Concurrency Problem: Lost Update (Overselling)

Explanation:

- Both transactions read the same stock value (1 ticket available).
- Both proceed to update it independently.
- One update overwrites the other, leading to incorrect inventory (selling more than available).

Summary

Scenario	Concurrency Problem	Key Issue
1	Non-Repeatable Read	Data changes during a transaction
2	Lost Update	Simultaneous updates cause incorrect results